Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЕТ

к лабораторной работе №4

на тему:

«АСИММЕТРИЧНАЯ КРИПТОГРАФИЯ. АЛГОРИТМ МАК-ЭЛИСА»

БГУИР 6-05-0612-02 74

Выполнил студент группы 353504
ЛЕВШУКОВ Дмитрий Александрович

Проверил ассистент каф. Информатики
ПОЛЯКОВ Кирилл Борисович

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Выполнение работы 4
Вывод 6
Приложение А (обязательное) Исходный код программы 7

ВВЕДЕНИЕ

McEliece – криптосистема с открытыми ключами на основе теории алгебраического кодирования, разработанная в 1978 году Робертом Мак-Элисом. Это была первая схема, использующая рандомизацию в процессе шифрования. Алгоритм основан на сложности декодирования полных линейных кодов.

Криптосистема имеет несколько преимуществ, например, над RSA. Шифрование и дешифрование проходит быстрее и с ростом длины ключа степень защиты данных растет гораздо быстрее. McEliece применим также в задачах аутентификации.

В системе Мак-Элиса параметрами системы, общими для всех абонентов, являются числа k, n, t. Для получения открытого и соответствующего секретного ключа каждому из абонентов системы следует осуществить следующие действия:

1 Выбрать порождающую матрицу $G = Gkn$ двоичного (n, k)-линейного кода, исправляющего t ошибок, для которого известен эффективный алгоритм декодирования.

2 Случайно выбрать двоичную невырожденную матрицу $S = Sk$.

3 Случайно выбрать подстановочную матрицу $P = Pn$.

4 Вычислить произведение матриц $G1 = S \times G \times P$.

Открытым ключом является пара $(G1, t)$, секретным – тройка (S, G, P).

Для того чтобы зашифровать сообщение M, предназначенное для абонента A, абоненту B следует выполнить следующие действия:

Представить M в виде двоичного вектора длины k.

Выбрать случайный бинарный вектор ошибок Z длиной n, содержащий не более t единиц.

Вычислить бинарный вектор $C = M \times GA + Z$ и направить его абоненту A.

Получив сообщение C, абонент A вычисляет вектор $C1 = C \times P - 1$, с помощью которого, используя алгоритм декодирования кода с порождающей матрицей G, получает далее векторы $M1$ и $M = M1 \times S - 1$.

В качестве кода, исправляющего ошибки в системе Мак-Элиса, можно использовать код Гоппы. Известно, что для любого неприводимого полинома $g(x)$ степени t над полем $GF(2m)$ существует бинарный код Гоппы длины $n = 2m$ и размерности $k \geq n - mt$, исправляющий до t ошибок включительно, для которого имеется эффективный алгоритм декодирования.

Задание:

1 Изучить теоретические сведения.

2 Реализовать криптостойкое программное средство шифрования и дешифрования текстовых файлов при помощи Криптосистемы Мак-Элиса.

1 ВЫПОЛНЕНИЕ РАБОТЫ

Общий алгоритм шифрования в криптосистеме Мак-Элиса представлен в виде блок-схемы на рисунке 1.1.

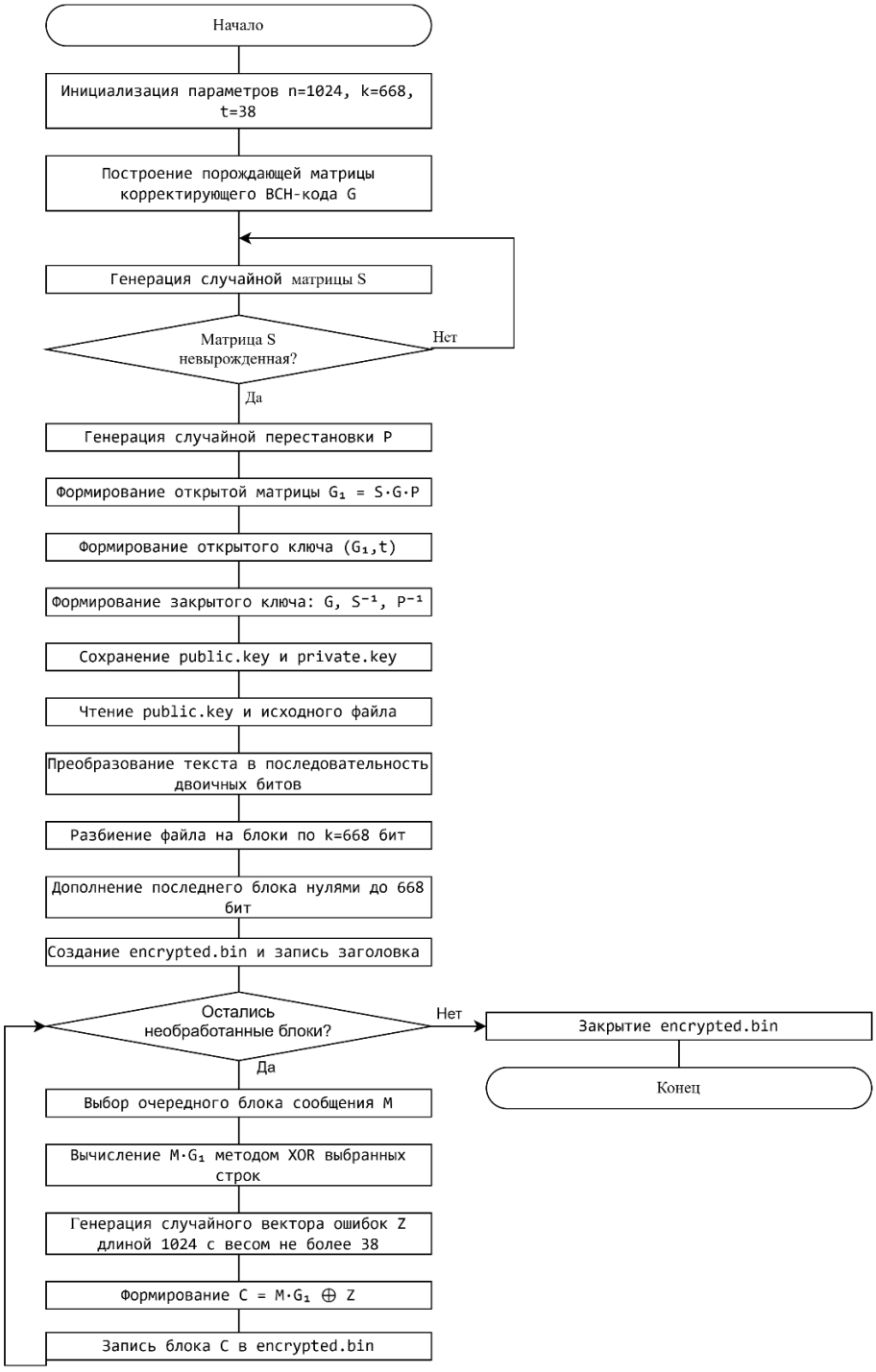


Рисунок 1.1 – Блок-схема алгоритма шифрования в криптосистеме Мак-Элиса

Для проверки работы разработанного алгоритма подается файл test.txt, содержимое которого представлено на рисунке 1.2.

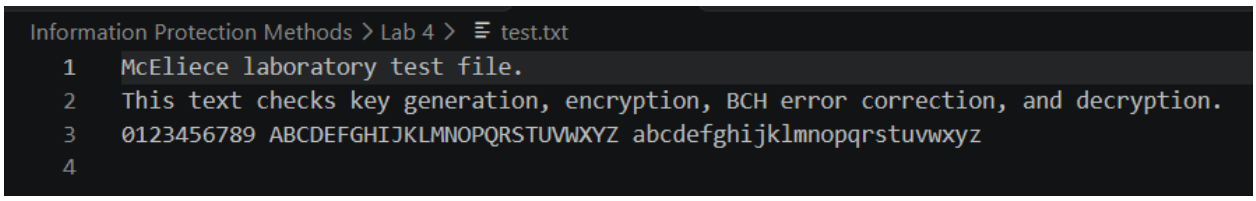


Рисунок 1.2 – Содержимое файла test.txt

Далее были сгенерированы открытый и закрытый ключи. Результат генерации открытого и закрытого ключей представлен на рисунке 1.3.



Рисунок 1.3 – Результат генерации открытого и закрытого ключей

Результат выполнения работы программы в режиме шифрования представлен на рисунке 1.4.



Рисунок 1.4 – Результат выполнения работы программы в режиме шифрования

Результат выполнения работы программы в режиме дешифрования представлен на рисунке 1.5.

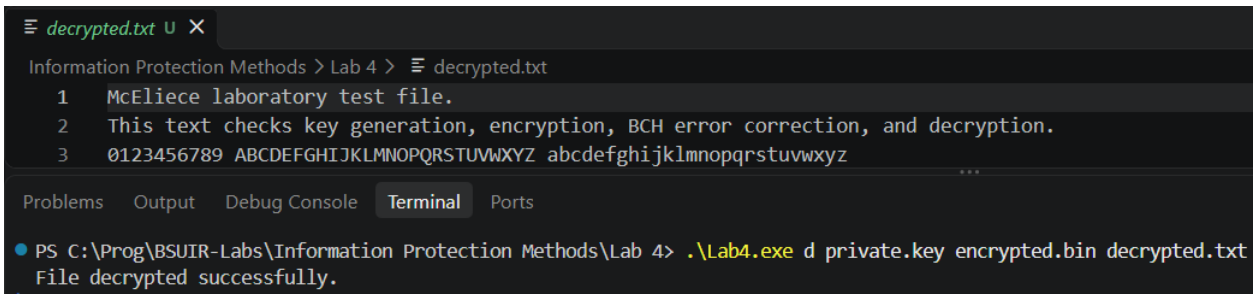


Рисунок 1.5 – Результат выполнения работы программы в режиме дешифрования

ВЫВОД

В ходе лабораторной работы была изучена и программно реализована криптосистема Мак-Элиса, основанная на использовании линейных кодов с исправлением ошибок.

В работе были рассмотрены структура открытого и закрытого ключей, построение порождающей матрицы BCH-кода, генерация случайной невырожденной матрицы и перестановочной матрицы, а также порядок выполнения шифрования и расшифрования сообщений.

Разработанная программа позволяет генерировать ключи, шифровать и расшифровывать текстовые файлы. При шифровании исходный файл преобразуется в последовательность двоичных битов и разбивается на блоки. К каждому блоку добавляется случайный вектор ошибок, который имитирует ошибки в передаваемом сообщении. При расшифровании эти ошибки обнаруживаются и исправляются с помощью BCH-кода, после чего исходное содержимое файла восстанавливается.

Корректность работы программы была проверена на тестовом текстовом файле. После шифрования и последующего расшифрования содержимое восстановленного файла полностью совпало с исходным.

Таким образом, в ходе лабораторной работы были закреплены навыки реализации криптографических алгоритмов, работы с бинарными матрицами, BCH-кодами, исправлением ошибок и обработки файлов на языке C++.

ПРИЛОЖЕНИЕ А
(обязательное)
Исходный код программы

```
#include <algorithm>
#include <array>
#include <cstdint>
#include <cstring>
#include <fstream>
#include <iostream>
#include <limits>
#include <numeric>
#include <stdexcept>
#include <string>
#include <unordered_set>
#include <utility>
#include <vector>

#include <windows.h>
#include <bcrypt.h>

constexpr std::size_t kCodeLength = 1024;
constexpr std::size_t kBchLength = 1023;
constexpr std::size_t kErrorCapacity = 38;
constexpr std::size_t kCodeWords = kCodeLength / 64;
constexpr char kPublicMagic[] = "MCELIECE-PUBLIC-1";
constexpr char kPrivateMagic[] = "MCELIECE-PRIVATE1";
constexpr char kCipherMagic[] = "MCELIECE-CIPHER-1";

using Word = std::uint64_t;

struct Block {
    std::array<Word, kCodeWords> words{};

    bool get(std::size_t bit) const {
        return ((words[bit / 64] >> (bit % 64)) & 1U) != 0;
    }

    void set(std::size_t bit, bool value) {
        const Word mask = Word{1} << (bit % 64);
        if (value) {
            words[bit / 64] |= mask;
        } else {
            words[bit / 64] &= ~mask;
        }
    }

    void xor_with(const Block& other) {
        for (std::size_t i = 0; i < words.size(); ++i) {
            words[i] ^= other.words[i];
        }
    }
};

void write_u32(std::ostream& out, std::uint32_t value) {
    out.write(reinterpret_cast<const char*>(&value), sizeof(value));
}

void write_u64(std::ostream& out, std::uint64_t value) {
    out.write(reinterpret_cast<const char*>(&value), sizeof(value));
}
```

```
      bool read_u32(std::istream& in, std::uint32_t& value) {
            return  static_cast<bool>(in.read(reinterpret_cast<char*>(&value),
sizeof(value)));
      }

      bool read_u64(std::istream& in, std::uint64_t& value) {
            return  static_cast<bool>(in.read(reinterpret_cast<char*>(&value),
sizeof(value)));
      }

      bool write_magic(std::ostream& out, const char* magic) {
          constexpr std::size_t max_magic_size = 32;
                          const         std::uint32_t         size         =
static_cast<std::uint32_t>(std::strlen(magic));
          if (size >= max_magic_size) {
              return false;
          }
          write_u32(out, size);
          out.write(magic, size);
          return static_cast<bool>(out);
      }

      bool read_magic(std::istream& in, const char* expected) {
          std::uint32_t size = 0;
          if (!read_u32(in, size) || size > 32) {
              return false;
          }
          std::string actual(size, '\0');
          if (!in.read(actual.data(), static_cast<std::streamsize>(size))) {
              return false;
          }
          return actual == expected;
      }

      bool secure_random_bytes(void* data, std::size_t size) {
                                       if                (size                >
static_cast<std::size_t>(std::numeric_limits<ULONG>::max())) {
              return false;
          }
          return BCryptGenRandom(nullptr, static_cast<PUCHAR>(data),
                                                  static_cast<ULONG>(size),
BCRYPT_USE_SYSTEM_PREFERRED_RNG) == 0;
      }

      std::uint64_t random_u64() {
          std::uint64_t value = 0;
          if (!secure_random_bytes(&value, sizeof(value))) {
              throw std::runtime_error("secure random generator failed");
          }
          return value;
      }

      std::uint32_t random_u32() {
          std::uint32_t value = 0;
          if (!secure_random_bytes(&value, sizeof(value))) {
              throw std::runtime_error("secure random generator failed");
          }
          return value;
      }

      class Gf1024 {
      public:
          static constexpr std::uint16_t primitive_polynomial = 0x409;
```

```
          Gf1024() {
              log_.fill(-1);
              std::uint16_t value = 1;
              for (std::size_t i = 0; i < kBchLength; ++i) {
                  exp_[i] = value;
                  log_[value] = static_cast<int>(i);
                  value = static_cast<std::uint16_t>(value << 1);
                  if ((value & 0x400U) != 0) {
                      value ^= primitive_polynomial;
                  }
              }
              for (std::size_t i = kBchLength; i < exp_.size(); ++i) {
                  exp_[i] = exp_[i - kBchLength];
              }
                primitive_ = value == 1 && std::all_of(log_.begin() + 1,
log_.end(),
                                                       [](int value) { return
value >= 0; });
          }

          bool primitive() const { return primitive_; }

          std::uint16_t multiply(std::uint16_t left, std::uint16_t right) const
{
              if (left == 0 || right == 0) {
                  return 0;
              }
              return exp_[static_cast<std::size_t>(log_[left] + log_[right])];
          }

          std::uint16_t inverse(std::uint16_t value) const {
              if (value == 0) {
                  return 0;
              }
              return exp_[kBchLength - static_cast<std::size_t>(log_[value])];
          }

          std::uint16_t alpha(std::size_t exponent) const {
              return exp_[exponent % kBchLength];
          }

      private:
          std::array<std::uint16_t, kBchLength * 2> exp_{};
          std::array<int, 1024> log_{};
          bool primitive_ = false;
      };

      struct CodeParameters {
          std::size_t n = kCodeLength;
          std::size_t t = kErrorCapacity;
          std::size_t r = 0;
          std::size_t k = 0;
          std::vector<std::uint8_t> generator;
          std::vector<Block> generator_rows;
      };

      std::vector<std::uint8_t> build_generator(const Gf1024& field) {
          std::vector<bool> visited(kBchLength, false);
          std::vector<std::uint16_t> polynomial{1};

           for (std::size_t exponent = 1; exponent <= 2 * kErrorCapacity;
++exponent) {
              if (visited[exponent]) {
                  continue;
```

```
            }

            std::vector<std::uint16_t> minimal{1};
            std::size_t current = exponent;
            do {
                visited[current] = true;
                const std::uint16_t root = field.alpha(current);
                std::vector<std::uint16_t> next(minimal.size() + 1, 0);
                for (std::size_t i = 0; i < minimal.size(); ++i) {
                    next[i] ^= field.multiply(minimal[i], root);
                    next[i + 1] ^= minimal[i];
                }
                minimal = std::move(next);
                current = (current * 2) % kBchLength;
            } while (current != exponent);

                    std::vector<std::uint16_t>   next(polynomial.size()   +
minimal.size() - 1, 0);
            for (std::size_t i = 0; i < polynomial.size(); ++i) {
                for (std::size_t j = 0; j < minimal.size(); ++j) {
                   next[i + j] ^= field.multiply(polynomial[i], minimal[j]);
                }
            }
            polynomial = std::move(next);
        }

        std::vector<std::uint8_t> generator(polynomial.size(), 0);
        for (std::size_t i = 0; i < polynomial.size(); ++i) {
            if (polynomial[i] == 0) {
                generator[i] = 0;
            } else if (polynomial[i] == 1) {
                generator[i] = 1;
            } else {
                  throw std::runtime_error("failed to construct binary BCH
generator");
            }
        }
        return generator;
    }

    CodeParameters make_code_parameters() {
        const Gf1024 field;
        if (!field.primitive()) {
            throw std::runtime_error("GF(2^10) polynomial is not primitive");
        }

        CodeParameters code;
        code.generator = build_generator(field);
        code.r = code.generator.size() - 1;
        code.k = kBchLength - code.r;
        if (code.k < 600 || code.k > 700 || code.generator.back() != 1) {
            throw std::runtime_error("unexpected BCH parameters");
        }

        code.generator_rows.resize(code.k);
        for (std::size_t row = 0; row < code.k; ++row) {
            Block block;
               for (std::size_t coefficient = 0; coefficient <= code.r;
++coefficient) {
                if (code.generator[coefficient] != 0) {
                    block.set(row + coefficient, true);
                }
            }
```

```
            bool parity = false;
            for (std::size_t bit = 0; bit < kBchLength; ++bit) {
                parity ^= block.get(bit);
            }
            block.set(kBchLength, parity);
            code.generator_rows[row] = block;
        }
        return code;
    }

    using Matrix = std::vector<std::vector<Word>>;

    std::size_t word_count(std::size_t bits) {
        return (bits + 63) / 64;
    }

    bool matrix_inverse(const Matrix& input, Matrix& inverse) {
        const std::size_t size = input.size();
        if (size == 0 || std::any_of(input.begin(), input.end(),
                                     [size](const auto& row) { return
row.size() != word_count(size); })) {
            return false;
        }

        const std::size_t words = word_count(size);
        Matrix augmented(size, std::vector<Word>(2 * words, 0));
        for (std::size_t row = 0; row < size; ++row) {
            for (std::size_t word = 0; word < words; ++word) {
                augmented[row][word] = input[row][word];
            }
            augmented[row][words + row / 64] |= Word{1} << (row % 64);
        }

        for (std::size_t column = 0; column < size; ++column) {
            std::size_t pivot = column;
            while (pivot < size &&
                   ((augmented[pivot][column / 64] >> (column % 64)) & 1U) ==
0) {
                ++pivot;
            }
            if (pivot == size) {
                return false;
            }
            if (pivot != column) {
                std::swap(augmented[pivot], augmented[column]);
            }

            for (std::size_t row = 0; row < size; ++row) {
                if (row == column ||
                    ((augmented[row][column / 64] >> (column % 64)) & 1U) ==
0) {
                    continue;
                }
                for (std::size_t word = 0; word < 2 * words; ++word) {
                    augmented[row][word] ^= augmented[column][word];
                }
            }
        }

        inverse.assign(size, std::vector<Word>(words, 0));
        for (std::size_t row = 0; row < size; ++row) {
            for (std::size_t word = 0; word < words; ++word) {
                inverse[row][word] = augmented[row][words + word];
            }
```

```
        }
        return true;
    }

    Matrix random_invertible_matrix(std::size_t size, Matrix& inverse) {
        const std::size_t words = word_count(size);
        Matrix matrix(size, std::vector<Word>(words));
        for (auto& row : matrix) {
            for (auto& value : row) {
                value = random_u64();
            }
            if ((size % 64) != 0) {
                row.back() &= (Word{1} << (size % 64)) - 1;
            }
        }
        while (!matrix_inverse(matrix, inverse)) {
            for (auto& row : matrix) {
                for (auto& value : row) {
                    value = random_u64();
                }
                if ((size % 64) != 0) {
                    row.back() &= (Word{1} << (size % 64)) - 1;
                }
            }
        }
        return matrix;
    }

    Block matrix_row_product(const std::vector<Word>& row, const
std::vector<Block>& rows,
                             std::size_t bits) {
        Block result;
        for (std::size_t bit = 0; bit < bits; ++bit) {
            if (((row[bit / 64] >> (bit % 64)) & 1U) != 0) {
                result.xor_with(rows[bit]);
            }
        }
        return result;
    }

    std::vector<std::uint32_t> random_permutation(std::size_t size) {
        std::vector<std::uint32_t> permutation(size);
        std::iota(permutation.begin(), permutation.end(), 0);
        for (std::size_t i = size; i > 1; --i) {
            const std::size_t j = random_u32() % i;
            std::swap(permutation[i - 1], permutation[j]);
        }
        return permutation;
    }

    struct PublicKey {
        std::uint32_t n = 0;
        std::uint32_t k = 0;
        std::uint32_t t = 0;
        std::vector<Block> rows;
    };

    struct PrivateKey {
        std::uint32_t n = 0;
        std::uint32_t k = 0;
        std::uint32_t t = 0;
        std::vector<std::uint8_t> generator;
        Matrix inverse_scrambler;
        std::vector<std::uint32_t> inverse_permutation;
```

```
};

bool write_public_key(const std::string& path, const PublicKey& key) {
    std::ofstream out(path, std::ios::binary);
    if (!out || !write_magic(out, kPublicMagic)) {
        return false;
    }
    write_u32(out, key.n);
    write_u32(out, key.k);
    write_u32(out, key.t);
    for (const auto& row : key.rows) {
        for (Word word : row.words) {
           out.write(reinterpret_cast<const char*>(&word), sizeof(word));
        }
    }
    return static_cast<bool>(out);
}

bool read_public_key(const std::string& path, PublicKey& key) {
    std::ifstream in(path, std::ios::binary);
    if (!in || !read_magic(in, kPublicMagic) || !read_u32(in, key.n) ||
        !read_u32(in, key.k) || !read_u32(in, key.t)) {
        return false;
    }
    if (key.n != kCodeLength || key.t != kErrorCapacity || key.k == 0 ||
key.k > 1024) {
        return false;
    }
    key.rows.assign(key.k, Block{});
    for (auto& row : key.rows) {
        for (Word& word : row.words) {
             if (!in.read(reinterpret_cast<char*>(&word), sizeof(word)))
{
                 return false;
            }
        }
    }
    return true;
}

bool write_private_key(const std::string& path, const PrivateKey& key) {
    std::ofstream out(path, std::ios::binary);
    if (!out || !write_magic(out, kPrivateMagic)) {
        return false;
    }
    write_u32(out, key.n);
    write_u32(out, key.k);
    write_u32(out, key.t);
    write_u32(out, static_cast<std::uint32_t>(key.generator.size()));
    out.write(reinterpret_cast<const char*>(key.generator.data()),
              static_cast<std::streamsize>(key.generator.size()));

    const std::size_t words = word_count(key.k);
    for (const auto& row : key.inverse_scrambler) {
        for (std::size_t word = 0; word < words; ++word) {
            const Word value = word < row.size() ? row[word] : 0;
                      out.write(reinterpret_cast<const   char*>(&value),
sizeof(value));
        }
    }
    for (std::uint32_t value : key.inverse_permutation) {
        write_u32(out, value);
    }
    return static_cast<bool>(out);
```

```
    }

    bool read_private_key(const std::string& path, PrivateKey& key) {
        std::ifstream in(path, std::ios::binary);
        std::uint32_t generator_size = 0;
        if (!in || !read_magic(in, kPrivateMagic) || !read_u32(in, key.n) ||
            !read_u32(in, key.k) || !read_u32(in, key.t) ||
            !read_u32(in, generator_size)) {
            return false;
        }
        if (key.n != kCodeLength || key.t != kErrorCapacity || key.k == 0 ||
key.k > 1024 ||
            generator_size == 0 || generator_size > kBchLength + 1) {
            return false;
        }
        key.generator.resize(generator_size);
              if    (!in.read(reinterpret_cast<char*>(key.generator.data()),
generator_size)) {
            return false;
        }

        const std::size_t words = word_count(key.k);
        key.inverse_scrambler.assign(key.k, std::vector<Word>(words, 0));
        for (auto& row : key.inverse_scrambler) {
            for (Word& value : row) {
               if (!in.read(reinterpret_cast<char*>(&value), sizeof(value)))
{
                    return false;
                }
            }
        }
        key.inverse_permutation.resize(key.n);
        for (auto& value : key.inverse_permutation) {
            if (!read_u32(in, value) || value >= key.n) {
                return false;
            }
        }
        return true;
    }

    bool generate_keys(const std::string& private_path, const std::string&
public_path,
                       const CodeParameters& code) {
        Matrix inverse_scrambler;
           const   Matrix   scrambler   =   random_invertible_matrix(code.k,
inverse_scrambler);
        const auto permutation = random_permutation(code.n);

        PublicKey public_key;
        public_key.n = static_cast<std::uint32_t>(code.n);
        public_key.k = static_cast<std::uint32_t>(code.k);
        public_key.t = static_cast<std::uint32_t>(code.t);
        public_key.rows.resize(code.k);
        for (std::size_t row = 0; row < code.k; ++row) {
              const Block transformed = matrix_row_product(scrambler[row],
code.generator_rows, code.k);
            Block permuted;
            for (std::size_t position = 0; position < code.n; ++position) {
                                                  permuted.set(position,
transformed.get(permutation[position]));
            }
            public_key.rows[row] = permuted;
        }
```

```
          PrivateKey private_key;
          private_key.n = public_key.n;
          private_key.k = public_key.k;
          private_key.t = public_key.t;
          private_key.generator = code.generator;
          private_key.inverse_scrambler = inverse_scrambler;
          private_key.inverse_permutation.resize(code.n);
          for (std::size_t position = 0; position < code.n; ++position) {
                    private_key.inverse_permutation[permutation[position]]   =
static_cast<std::uint32_t>(position);
          }

          return write_private_key(private_path, private_key) &&
                 write_public_key(public_path, public_key);
      }

      std::uint16_t  evaluate_syndrome(const  Gf1024&  field,  const  Block&
received,
                                       std::size_t syndrome_number) {
          std::uint16_t result = 0;
          for (std::size_t position = 0; position < kBchLength; ++position) {
              if (received.get(position)) {
                       result  ^=  field.alpha((syndrome_number  *  position)  %
kBchLength);
              }
          }
          return result;
      }

      bool  decode_bch(const  Gf1024&  field,  const  std::vector<std::uint8_t>&
generator,
                    Block& received, std::size_t t, std::vector<std::uint8_t>&
message) {
          std::vector<std::uint16_t> syndromes(2 * t, 0);
          bool has_error = false;
          for (std::size_t i = 0; i < syndromes.size(); ++i) {
              syndromes[i] = evaluate_syndrome(field, received, i + 1);
              has_error = has_error || syndromes[i] != 0;
          }

          if (has_error) {
              std::vector<std::uint16_t> locator(2 * t + 1, 0);
              std::vector<std::uint16_t> backup(2 * t + 1, 0);
              locator[0] = backup[0] = 1;
              std::size_t degree = 0;
              std::size_t shift = 1;
              std::uint16_t last_discrepancy = 1;

              for (std::size_t n = 0; n < 2 * t; ++n) {
                  std::uint16_t discrepancy = syndromes[n];
                  for (std::size_t i = 1; i <= degree; ++i) {
                       discrepancy ^= field.multiply(locator[i], syndromes[n -
i]);
                  }

                  if (discrepancy == 0) {
                      ++shift;
                      continue;
                  }

                  const auto previous = locator;
                  const std::uint16_t scale = field.multiply(discrepancy,
                                                             field.inverse(la
st_discrepancy));
```

```
                for (std::size_t i = 0; i + shift < locator.size(); ++i) {
                    locator[i + shift] ^= field.multiply(scale, backup[i]);
                }
                if (2 * degree <= n) {
                    degree = n + 1 - degree;
                    backup = previous;
                    last_discrepancy = discrepancy;
                    shift = 1;
                } else {
                    ++shift;
                }
            }

            if (degree == 0 || degree > t) {
                return false;
            }

            std::size_t roots = 0;
            for (std::size_t position = 0; position < kBchLength; ++position)
{
                const std::uint16_t point = field.alpha(position == 0 ? 0 :
kBchLength - position);
                std::uint16_t value = 0;
                std::uint16_t power = 1;
                for (std::size_t i = 0; i <= degree; ++i) {
                    value ^= field.multiply(locator[i], power);
                    power = field.multiply(power, point);
                }
                if (value == 0) {
                    received.set(position, !received.get(position));
                    ++roots;
                }
            }
            if (roots != degree) {
                return false;
            }

            for (std::size_t i = 0; i < syndromes.size(); ++i) {
                if (evaluate_syndrome(field, received, i + 1) != 0) {
                    return false;
                }
            }
        }

        const std::size_t r = generator.size() - 1;
        const std::size_t k = kBchLength - r;
        std::vector<std::uint8_t> polynomial(kBchLength, 0);
        for (std::size_t i = 0; i < kBchLength; ++i) {
            polynomial[i] = received.get(i) ? 1 : 0;
        }

        message.assign(k, 0);
        for (std::size_t position = kBchLength; position-- > r;) {
            if (polynomial[position] == 0) {
                continue;
            }
            message[position - r] = 1;
            for (std::size_t coefficient = 0; coefficient <= r; ++coefficient)
{
                polynomial[position - r + coefficient] ^=
generator[coefficient];
            }
        }
        for (std::size_t i = 0; i < r; ++i) {
```

```
            if (polynomial[i] != 0) {
                return false;
            }
        }
        return true;
    }

    bool read_file(const std::string& path, std::vector<std::uint8_t>& data)
{
        std::ifstream in(path, std::ios::binary);
        if (!in) {
            return false;
        }
        in.seekg(0, std::ios::end);
        const auto end = in.tellg();
        if (end < 0) {
            return false;
        }
        data.resize(static_cast<std::size_t>(end));
        in.seekg(0, std::ios::beg);
                                return             data.empty()             ||
static_cast<bool>(in.read(reinterpret_cast<char*>(data.data()),
                                                              static_cast<std::st
reamsize>(data.size())));
    }

    bool  encrypt_file(const  std::string&  public_path,  const  std::string&
input_path,
                       const std::string& output_path) {
        PublicKey key;
        std::vector<std::uint8_t> input;
         if  (!read_public_key(public_path,  key)  ||  !read_file(input_path,
input)) {
            return false;
        }

        const std::size_t capacity_bits = static_cast<std::size_t>(key.k);
        const std::uint64_t block_count =
            input.empty() ? 0 : (static_cast<std::uint64_t>(input.size()) *
8 + capacity_bits - 1) /
                                    capacity_bits;
        std::ofstream out(output_path, std::ios::binary);
        if (!out || !write_magic(out, kCipherMagic)) {
            return false;
        }
        write_u32(out, key.n);
        write_u32(out, key.k);
        write_u32(out, key.t);
        write_u64(out, static_cast<std::uint64_t>(input.size()));
        write_u64(out, block_count);

        std::size_t source_bit = 0;
         for  (std::uint64_t  block_index  =  0;  block_index  <  block_count;
++block_index) {
            Block encrypted;
           for (std::size_t bit = 0; bit < capacity_bits; ++bit, ++source_bit)
{
                if (source_bit < input.size() * 8 &&
                    ((input[source_bit / 8] >> (source_bit % 8)) & 1U) != 0)
{
                    encrypted.xor_with(key.rows[bit]);
                }
            }
```

```
            const std::size_t error_count = random_u32() % (key.t + 1);
            std::unordered_set<std::size_t> positions;
            while (positions.size() < error_count) {
                positions.insert(random_u32() % key.n);
            }
            for (std::size_t position : positions) {
                encrypted.set(position, !encrypted.get(position));
            }

            for (Word word : encrypted.words) {
               out.write(reinterpret_cast<const char*>(&word), sizeof(word));
            }
        }
        return static_cast<bool>(out);
    }

    bool decrypt_file(const std::string& private_path, const std::string&
input_path,
                      const std::string& output_path, const CodeParameters&
code) {
        PrivateKey key;
        std::ifstream in(input_path, std::ios::binary);
        if (!in || !read_magic(in, kCipherMagic)) {
            return false;
        }
        std::uint32_t n = 0;
        std::uint32_t k = 0;
        std::uint32_t t = 0;
        std::uint64_t original_size = 0;
        std::uint64_t block_count = 0;
        if (!read_u32(in, n) || !read_u32(in, k) || !read_u32(in, t) ||
            !read_u64(in, original_size) || !read_u64(in, block_count) ||
            !read_private_key(private_path, key)) {
            return false;
        }
        if (n != key.n || k != key.k || t != key.t || n != kCodeLength ||
            k != code.k || t != kErrorCapacity ||
            block_count != (original_size == 0 ? 0 : (original_size * 8 + k
- 1) / k)) {
            return false;
        }
                                  if             (original_size             >
static_cast<std::uint64_t>(std::numeric_limits<std::size_t>::max())) {
            return false;
        }

        const Gf1024 field;
        const std::size_t words = word_count(key.k);
        std::vector<Block> inverse_rows(key.k);
        for (std::size_t row = 0; row < key.k; ++row) {
            for (std::size_t column = 0; column < key.k; ++column) {
                 if (((key.inverse_scrambler[row][column / 64] >> (column %
64)) & 1U) != 0) {
                    inverse_rows[row].set(column, true);
                }
            }
        }
                                                  std::vector<std::uint8_t>
output(static_cast<std::size_t>(original_size), 0);
        std::size_t destination_bit = 0;
         for (std::uint64_t block_index = 0; block_index < block_count;
++block_index) {
            Block received;
            for (Word& word : received.words) {
```

```
                    if (!in.read(reinterpret_cast<char*>(&word), sizeof(word)))
{
                        return false;
                    }
                }

                Block codeword;
                for (std::size_t position = 0; position < key.n; ++position) {
                                                        codeword.set(position,
received.get(key.inverse_permutation[position]));
                }

                std::vector<std::uint8_t> scrambled_message;
                     if  (!decode_bch(field,  key.generator,  codeword,  key.t,
scrambled_message) ||
                    scrambled_message.size() != key.k) {
                    return false;
                }
                std::vector<Word> message_row(words, 0);
                for (std::size_t bit = 0; bit < key.k; ++bit) {
                    if (scrambled_message[bit] != 0) {
                        message_row[bit / 64] |= Word{1} << (bit % 64);
                    }
                }
                     const  auto  plain_row  =  matrix_row_product(message_row,
inverse_rows, key.k);

                  for (std::size_t bit = 0; bit < key.k && destination_bit <
output.size() * 8;
                       ++bit, ++destination_bit) {
                    if (plain_row.get(bit)) {
                      output[destination_bit / 8] |= static_cast<std::uint8_t>(1U
<< (destination_bit % 8));
                    }
                }
            }

            std::ofstream out(output_path, std::ios::binary);
            if (!out) {
                return false;
            }
            if (!output.empty()) {
                out.write(reinterpret_cast<const char*>(output.data()),
                          static_cast<std::streamsize>(output.size()));
            }
            return static_cast<bool>(out);
        }

        void print_usage(const char* program) {
            std::cerr << "Usage:\n"
                      << "  " << program << " keygen <private_key> <public_key>\n"
                       << "  " << program << " encrypt|e <public_key> <input_file>
<output_file>\n"
                      << "  " << program << " decrypt|d <private_key> <input_file>
<output_file>\n";
        }

        int main(int argc, char* argv[]) {
            try {
                if (argc < 2 || argc > 5) {
                    print_usage(argv[0]);
                    return 1;
                }
```

```
            const std::string command = argv[1];
            const CodeParameters code = make_code_parameters();

            if (command == "keygen") {
                if (argc != 4) {
                    print_usage(argv[0]);
                    return 1;
                }
                if (!generate_keys(argv[2], argv[3], code)) {
                    std::cerr << "Error: failed to write key files\n";
                    return 1;
                }
                std::cout << "Keys generated successfully.\n"
                          << "Parameters: n=" << code.n << ", k=" << code.k
                          << ", t=" << code.t << "\n";
                return 0;
            }

            if (command == "encrypt" || command == "e") {
                if (argc != 5 || !encrypt_file(argv[2], argv[3], argv[4])) {
                    std::cerr << "Error: encryption failed\n";
                    return 1;
                }
                std::cout << "File encrypted successfully.\n";
                return 0;
            }

            if (command == "decrypt" || command == "d") {
                 if (argc != 5 || !decrypt_file(argv[2], argv[3], argv[4],
code)) {
                    std::cerr << "Error: decryption failed or ciphertext is
invalid\n";
                    return 1;
                }
                std::cout << "File decrypted successfully.\n";
                return 0;
            }

            print_usage(argv[0]);
            return 1;
        } catch (const std::exception& error) {
            std::cerr << "Error: " << error.what() << "\n";
            return 1;
        }
    }
```